

Sprint Action Plan: Person 5: Baselines Lead

SOTA Recurrent Baselines & Gated DeltaNet Implementation • Adaptive Attention Computation Sprint

Assigned Role: Person 5: Baselines Lead	Estimated Workload: 8.5 Hours	Sprint Timeline: 0.0h – 9.5h
Core Discipline: Recurrent Transformer Architecture / State-Space Models	Primary Handoff To: Person 6 (Synthesis Lead)	Target Hardware: Dual T4 (16GB)

1. Role Mission & Strategic Objective

Implement a pure-PyTorch layer for Gated DeltaNet to evaluate modern error-correcting recurrent mechanisms. Benchmark throughput, recurrent state retention, and memory scaling against standard linear attention on Kaggle T4.

2. Input Dependencies & Final Deliverables

Type	Item / Specification	Source / Destination	Deadline / Gate
INPUT	Gated DeltaNet mathematical formulation	Literature / Project Specification	Hour 0.0
INPUT	Frozen CSV schema contract	Person 6 (Synthesis Lead)	Hour 1.0
OUTPUT	DeltaNet benchmark notebook (03_DeltaNet_T4.ipynb)	Team Repository	Hour 7.5
OUTPUT	Comparative baseline metrics table (deltanet_comparison.csv)	Person 6 (Synthesis Lead)	Hour 8.0
OUTPUT	Slide 7 SOTA baseline summary notes	Person 6 (Synthesis Lead)	Hour 8.5

3. Technical Specifications & Contract Signature

Key Code / Mathematical Contract for Person 5: Baselines Lead:

- Delta Update Rule: $S_t = S_{t-1} + \beta_t * (v_t - S_{t-1} k_t) * k_t^T$
- Gating Mechanism: Data-dependent decay gate $\alpha_t \in (0, 1)$ ensuring stable memory retention
- Implementation Rule: Pure PyTorch layer (avoid custom Triton/C++ extensions to ensure Kaggle compatibility)
- Benchmarking Regimes: Profile latency and recurrent memory across $N \in \{1024, 2048, 4096, 8192, 16384\}$

4. Hour-by-Hour Master Action Checklist — Person 5: Baselines Lead

Window	Specific Actions & Step-by-Step Instructions	Deliverable / Evidence	Status
0.0 – 1.0 h	Extract Gated DeltaNet recurrent mathematics and define tensor dimension contracts.	DeltaNet mathematical specification	[] Done
1.0 – 3.0 h	Implement standalone pure-PyTorch layer for Gated DeltaNet with data-dependent update gates.	Standalone DeltaNet PyTorch layer	[] Done
3.0 – 5.5 h	Profile DeltaNet throughput and execution latency on Kaggle T4 across N = 1K to 16K.	DeltaNet throughput profiling data	[] Done
5.5 – 7.5 h	Profile recurrent state memory footprint vs Linear Attention; evaluate state retention on associative recall.	State retention & memory comparison	[] Done
7.5 – 8.5 h	Finalize comparative baseline metrics table; export deltanet_comparison.csv to Person 6.	deltanet_comparison.csv delivered to P6	[] Done
8.5 – 9.5 h	Audit comparative SOTA claims in Slide 7; participate in presentation rehearsal.	Slide 7 audit sign-off	[] Done

5. Operational Guardrails & Failure Protocols

Constraint / Risk	Enforcement Rule & Exact Action
Compilation Failures	Strictly implement DeltaNet in pure PyTorch (torch.matmul / einsum) without custom CUDA extensions.
Recurrent Loop Latency	Use chunked associative scans where possible or batch sequence iterations to optimize T4 throughput.
Numerical Divergence	Bound delta updates with sigmoid / tanh activations on beta gates to avoid exploding state norms.

6. Definition of Done (DoD) & Sign-Off Checklist

[]	Standalone Gated DeltaNet layer functions cleanly in pure PyTorch.
[]	Throughput benchmarked up to N = 16,384 on Tesla T4.
[]	deltanet_comparison.csv populated with latency, memory, and state retention metrics.
[]	Comparative analysis delivered to Person 6 for Slide 7.